Pontifícia Universidade Católica de Minas Gerais

Disciplina: Teste de Software

Detecção de Bad Smells e Refatoração Segura

Aluno: Vitor Costa Vianna

Matrícula: 857863

Professor: Cleiton Tavares

Repositório: github.com/VitorCostaVianna/bad-smells-js-refactoring

1. Análise de Smells

O arquivo `src/ReportGenerator.js` tem um único método, `generateReport(reportType, user, items)`, que gera relatórios em CSV ou HTML e filtra os itens pelo papel do usuário (ADMIN vê tudo, USER só vê itens com valor ≤ 500). A análise seguiu o catálogo de Fowler visto em aula, e os três smells abaixo foram escolhidos por serem os mais graves.

1.1 Long Method / God Method

O método tem 58 linhas e concentra todo o comportamento da classe: cabeçalho, regra de visibilidade por papel, prioridade, formatação das linhas nos dois formatos, total e rodapé, com quatro níveis de aninhamento (`for` → `if role` → `if value` → `if reportType`). **Manutenção:** qualquer alteração exige ler o método inteiro e arrisca afetar as outras responsabilidades. **Testes:** nenhuma regra pode ser testada isoladamente. Para verificar só o limite de 500, é preciso gerar o relatório completo e procurar texto na saída, e os caminhos (formato × papel × faixa de valor) se multiplicam. Refatorações do catálogo: Extract Method e Replace Method with Method Object.

1.2 Switch Statements

A cadeia `if (reportType === 'CSV') … else if (reportType === 'HTML')` se repete quatro vezes (linhas 16, 35, 46 e 58), e há outra cadeia sobre `user.role`. **Manutenção:** um novo formato (ex.: JSON) obriga a alterar os quatro pontos, e esquecer um deles gera um relatório inconsistente sem nenhum erro. **Testes:** cada novo formato precisa ser testado para cada papel, o que multiplica os cenários. Refatorações do catálogo: Extract Method e Replace Conditional with Polymorphism.

1.3 Duplicated Code

A linha CSV é idêntica nas linhas 36 e 47, a linha HTML é quase igual nas linhas 40 e 50, e `total += item.value` aparece quatro vezes. **Manutenção:** as cópias divergem com o tempo, e isso já aconteceu aqui: o ramo do ADMIN gera `<tr >` (com espaço) para itens sem prioridade, enquanto o do USER gera `<tr>`. Por isso o teste *"deve gerar um relatório HTML completo para Admin"* já falhava no código original. **Testes:** cada cópia precisa do seu próprio caso de teste, e testar o USER não garante nada sobre o ADMIN. Refatoração do catálogo: Extract Method.

Do mesmo catálogo, também foram encontrados: **Divergent Change** (a classe muda por motivos diferentes: novo formato, regra de papel, layout), **Feature Envy** (as decisões dependem de `user.role` e `item.value`, dados de outros objetos) e **Comments** usados como desodorante (`// --- Seção do Cabeçalho ---`). Fora do catálogo, o código ainda tem os números mágicos 500 e 1000 e um efeito colateral: `item.priority = true` altera o objeto recebido.

2. Relatório da Ferramenta

O `.eslintrc.json` estende `eslint:recommended` e `plugin:sonarjs/recommended`, com o limite de `sonarjs/cognitive-complexity` reduzido para 5.

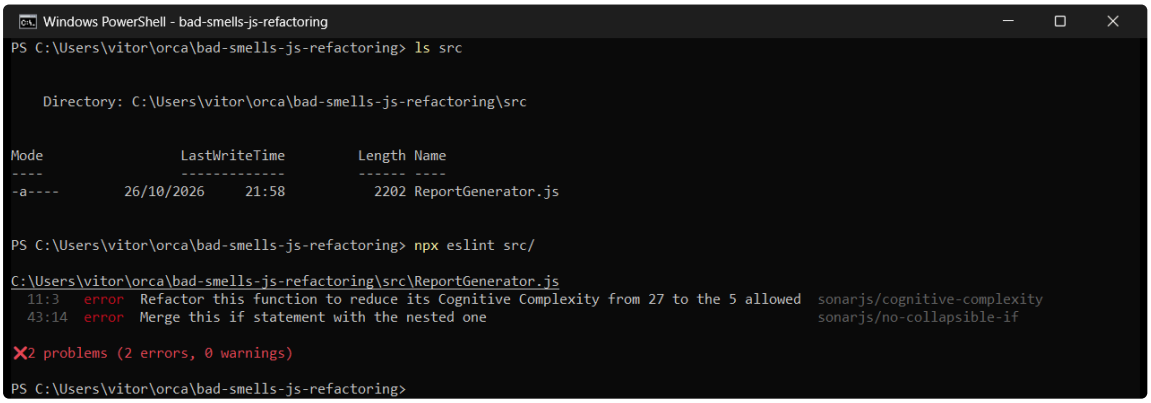


Figura 1 – **Antes** da refatoração: `src/` contém só o código original, e `npx eslint src/` aponta 2 erros.

O `eslint-plugin-sonarjs` transformou uma impressão subjetiva em uma medida. Ao ler o código, percebe-se que o método é "grande", mas não o quanto. A **complexidade cognitiva** dá um número: cada `if`, `else if` e laço soma pontos, e cada nível de aninhamento soma mais. O valor 27, contra um limite de 5, mostra que o problema principal é o aninhamento das decisões, e não a quantidade de linhas. Isso indicou que a solução era desfazer o aninhamento, e não só quebrar o método em pedaços. A regra `no-collapsible-if` (linha 43) apontou um detalhe fácil de ignorar numa leitura rápida: `else if (role === 'USER') { if (value <= 500) … }` são dois níveis onde

bastaria um. A ferramenta também tem limites: a regra `no-duplicated-branches` estava ativa e não detectou a duplicação das linhas 36/47 e 40/50, porque as cópias estão em `if`s diferentes. Duplicated Code, Divergent Change e Feature Envy só apareceram na análise manual. Uma complementa a outra.

3. Processo de Refatoração

O smell mais crítico corrigido foi o **Long Method**, que, junto aos **Switch Statements** repetidos, explica os 27 pontos de complexidade. A Figura 2 compara o original com `src/ReportGenerator.refactored.js`.

Antes: corpo de `generateReport` (trecho)

```
for (const item of items) {
  if (user.role === 'ADMIN') {
    // Admins veem todos os itens
    if (item.value > 1000) {
      item.priority = true;
    }
    if (reportType === 'CSV') {
      report += `${item.id},${item.name},…,${user.name}\n`;
      total += item.value;
    } else if (reportType === 'HTML') {
      const style = item.priority
        ? 'style="font-weight:bold;"' : '';
      report += `<tr ${style}><td>${item.id}</td>…</tr>\n`;
      total += item.value;
    }
  } else if (user.role === 'USER') {
    // Users comuns só veem itens de valor baixo
    if (item.value <= 500) {
      if (reportType === 'CSV') {
        report += `${item.id},${item.name},…,${user.name}\n`;
        total += item.value;
      } else if (reportType === 'HTML') {
        report += `<tr><td>${item.id}</td>…</tr>\n`;
        total += item.value;
      }
    }
  }
}
// + cabeçalho e rodapé: mais dois if CSV/HTML
```

Depois: `ReportGenerator.refactored.js` (trecho)

```
class HtmlFormatter {
  header(user) { /* … */ }
  row(item, user, isPriority) {
    const style = isPriority
      ? ' style="font-weight:bold;"' : '';
    return `<tr${style}><td>${item.id}</td>…</tr>\n`;
  }
  footer(total) { /* … */ }
}
// CsvFormatter: mesma interface

const ROLE_POLICIES = new Map([
  ['ADMIN', { canView: () => true,
    isPriority: (item) => Boolean(item.priority)
      || item.value > PRIORITY_THRESHOLD }],
  ['USER', { canView: (item) =>
      item.value <= USER_MAX_VISIBLE_VALUE,
    isPriority: () => false }],
]);

generateReport(reportType, user, items) {
  const formatter = FORMATTERS.get(reportType);
  if (!formatter) return '';
  const policy =
    ROLE_POLICIES.get(user.role) ?? NO_ACCESS_POLICY;
  const visibleItems =
    items.filter((item) => policy.canView(item));
  const rows = visibleItems
    .map((item) =>
      formatter.row(item, user, policy.isPriority(item)))
    .join('');
  return (formatter.header(user) + rows
    + formatter.footer(sumValues(visibleItems))).trim();
}
```

Figura 2 – Antes e depois, em trechos resumidos (… indica partes omitidas, e algumas linhas foram quebradas para caber). Em destaque, a linha duplicada que divergiu e gerava `<tr >`.

Técnicas aplicadas, com os nomes do catálogo visto em aula:

- **Extract Method:** usei para transformar cada seção comentada (cabeçalho, linha, rodapé) em um método próprio, `header()`, `row()` e `footer()`, e para calcular o total uma única vez em `sumValues()`. Isso eliminou o código duplicado e os comentários-desodorante.
- **Extract Class:** usei para tirar a formatação de dentro de `ReportGenerator`. Os métodos extraídos foram para as novas classes `CsvFormatter` e `HtmlFormatter`, o que resolve o Divergent Change: mudar um formato não toca mais na regra de negócio.
- **Replace Conditional with Polymorphism:** usei para eliminar as quatro cadeias CSV/HTML. As duas classes têm a mesma interface, e o formatador certo é escolhido uma única vez em `FORMATTERS`. Um novo formato passa a ser só uma nova classe. Apliquei a mesma ideia aos papéis: os `if`s sobre `user.role` viraram objetos com a mesma interface (`canView`, `isPriority`) em `ROLE_POLICIES`, o que também eliminou o `no-collapsible-if`.
- **Move Method:** usei para levar as decisões que "invejavam" os dados de `user` e `item` (quem vê o quê, o que é prioritário) para as políticas de cada papel, resolvendo o Feature Envy.

Além disso, os números mágicos viraram as constantes `USER_MAX_VISIBLE_VALUE` e `PRIORITY_THRESHOLD`, e o item recebido não é mais alterado.

O resultado é um `generateReport` linear, de 15 linhas: escolher o formatador e a política, filtrar, formatar e somar. A maior complexidade cognitiva do arquivo caiu de 27 para 2.

4. Validação com a Rede de Segurança

O teste foi duplicado em `tests/ReportGenerator.refactored.test.js`, e só o `import` mudou. O enunciado cita a pasta `__tests__/`, mas o repositório base guarda o teste original em `tests/`, então a cópia ficou ao lado dele. O script `test` passou a usar `node --experimental-vm-modules`, porque sem essa flag o Jest não executa nenhum teste em ES Modules. A primeira execução revelou o defeito do `<tr >` no original, que corrigi com uma alteração de uma linha, sem mexer em nenhum teste. Além da suíte, comparei a saída dos dois arquivos em 40 combinações de entrada (formatos e papéis válidos e inválidos, valores 500/1000/1001 e lista vazia), e as saídas foram idênticas. A única mudança de comportamento é intencional: o refatorado não altera mais os itens recebidos.

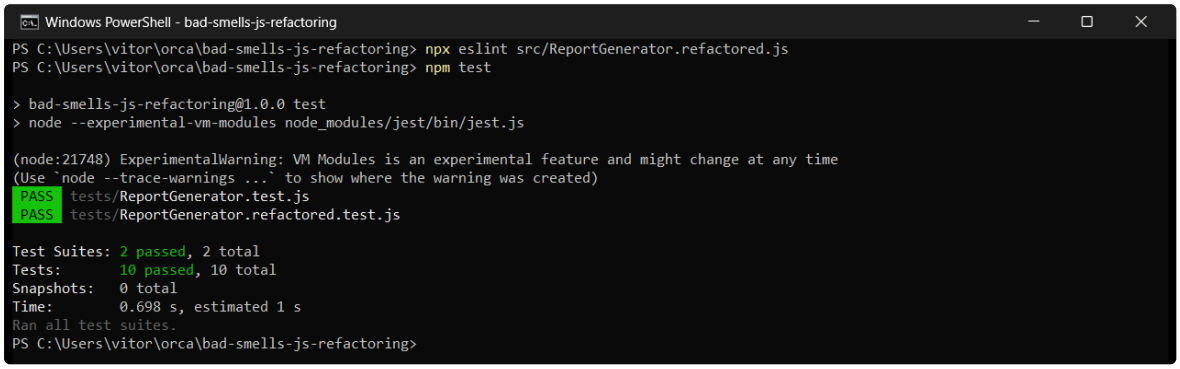


Figura 3 – **Depois** da refatoração: `npx eslint` não aponta nenhum erro no arquivo refatorado, e `npm test` passa as duas suítes (10/10).

Métrica	Original	Refatorado
Erros do ESLint	2	0
Complexidade cognitiva máx.	27	2
Linhas de `generateReport`	58	15
Decisões sobre o formato	4	1
Testes passando	5/5*	5/5

* após corrigir o `<tr >`. Antes, 4/5.

5. Conclusão

Refatorar sem testes é uma aposta. Aqui a suíte funcionou como rede de segurança antes mesmo da primeira mudança: ela revelou um defeito real no original, causado justamente pela duplicação. Durante a refatoração, cada passo foi validado contra o mesmo comportamento esperado, o que permitiu reestruturar o método por completo (novas classes, polimorfismo, outra forma de calcular o total) com a confiança de que os relatórios gerados continuaram os mesmos. Isso só foi possível porque os testes verificam a saída e não a implementação. A redução dos smells melhorou a qualidade de forma concreta. O código ficou mais legível (15 linhas lineares em vez de 58 aninhadas) e mais fácil de estender, já que um formato novo é uma classe nova. Também ficou mais testável: como as regras de papel e a formatação agora estão separadas, cada parte pode passar a ter testes próprios, sem depender de gerar o relatório inteiro. A ferramenta e a leitura humana se complementam: o `sonarjs` mede a complexidade com precisão, mas foi a análise manual que encontrou a duplicação e os problemas de design.